THREADS

Devices

Devices Project Definition

(Spring 2026)

Version 1.1

1. Introduction

The fourth stage of our OS development project extends your kernel with support for a clock driver and for disk input/output (I/O) operations. This functionality is exposed through four new system calls: **`SleepSeconds, DiskRead, DiskWrite, and DiskInfo`**. These system calls are invoked just like the ones implemented in the System Calls project — by triggering the syscall interrupt, which dispatches control to your system call handler within the Devices implementation.

To support these new system calls, your kernel will introduce two types of internal driver processes: the *Clock Driver* and the *Disk Driver*. These driver processes run in kernel mode, with interrupts enabled, and handle key asynchronous behavior in the system.

- The Clock Driver implements the behavior of Sleep.
- The Disk Driver is responsible for managing disk I/O requests. Importantly, there will be *two* instances of the Disk Driver process — one dedicated to each disk in the system.

Each Disk Driver manages requests for its assigned disk independently, allowing multiple I/O operations to proceed concurrently across devices while still preserving correct sequencing and synchronization for individual disks.

1.1. Objectives and Scope

The objective of this document is to specify requirements for the Devices development project. Included is a specification for the required functions (e.g. the Devices Application Programming Interface (API)) that must be implemented for this part of the kernel's development.
This document combined with the course's lecture and presentation materials define the totality of the Devices project.

1.2. Target Audience

The document is intended for students taking advanced operating systems courses at the graduate or undergraduate level.

2. High Level Overview

The Devices project expands your kernel to include new functionality for managing time and handling disk I/O. This is accomplished through the addition of new devices, internal kernel processes, and interrupt handling routines that work together to provide asynchronous services to user processes.

The *Clock Driver* is a kernel process responsible managing sleeping processes. When a user process invokes the **`SleepSeconds`** system call, the Clock Driver adds that process to a list of sleeping processes, along with the target wake-up time. As timer interrupts occur and time

advances, the Clock Driver checks this list and unblocks any processes whose sleep duration has expired.

Disk I/O is managed by a separate *Disk Driver* process for each disk device in the system. There are two disks, and therefore two instances of the Disk Driver — each responsible for a single disk. When a user process invokes the **`DiskRead`** or **`DiskWrite`** system calls, the request is validated and forwarded to the appropriate Disk Driver. The driver queues the request and interacts with the disk device to initiate the operation. Because disk operations are asynchronous, the calling process is suspended until the I/O completes. Once the device signals completion through an interrupt, the Disk Driver unblocks the waiting process so that it can resume execution.

In addition to handling read and write operations, each Disk Driver supports the **`DiskInfo`** system call. This call allows user processes to query the geometry of the disk, including the number of sectors, tracks, and platters.

All of these features are integrated through an extension of the system call handler developed in the System Calls project. If you need to review how system calls are configured, dispatched, and handled, refer back to the System Calls project materials for guidance.

We are implementing the system calls defined in Table 1.

Table 1 System calls implemented in the Devices project.

System Call Id	User Process Interface
SYS_SLEEP	SleepSeconds
SYS_DISKINFO	DiskInfo
SYS_DISKREAD	DiskRead
SYS_DISKWRITE	DiskWrite

3. Clock Management

Timing management in this project is handled by a dedicated Clock Driver process. The Clock Driver is a kernel-mode process that is responsible for implementing the operating logic behind the **`SleepSeconds`** system call by managing the waking of processes that have requested to sleep.

The Clock Driver interacts with the timer created during the Messaging project by calling the **`waitdevice`** function. The timer device signals the Clock Driver's mailbox every 5 timer interrupt occurrences. After returning from **`waitdevice`** — triggered by receiving a message from the timer device — the Clock Driver checks for any processes whose sleep duration has expired.

User processes request to sleep by invoking the **`SleepSeconds`** system call. The system call handler is responsible for converting the requested sleep duration from seconds into an appropriate number of microseconds. The **`system_clock`**() function provides the current system time in microseconds since system startup, allowing for accurate calculations. Once the sleep

duration has been calculated, the calling process is added to a sleep list managed by the Clock Driver. The process is then blocked until the Clock Driver determines that it is time for the process to resume.

The sleep list contains all processes that are currently sleeping, along with the target wake-up time for each. This wake-up time is expressed in microseconds and represents the value of **`system_clock()`** at which the process should be unblocked. The wake-up time is calculated as the current value of **`system_clock()`** plus the requested sleep duration converted to microseconds (seconds × 1,000,000).

When the Clock Driver is awakened by a timer event every 5 timer interrupt occurrences, it checks the sleep list for any processes whose wake-up time has arrived or passed. Any such processes are unblocked so that they may resume execution. The Clock Driver then returns to waiting on **`waitdevice`** for the next timer interrupt.

This design ensures that the Clock Driver only executes in response to timer events and that processes sleeping for relatively short or long durations are all handled using the same mechanism. The driver does not increment time itself but relies on either the system-provided **`system_clock`**() function or its own internal tracking of time based on the number of interrupts it has received.

Processes invoking **`SleepSeconds`** are suspended until the Clock Driver explicitly unblocks them when their requested sleep time has expired.

4. Disk Management

Disk I/O in this project is managed by a dedicated Disk Driver process for each disk device in the system. There are two disk devices, and therefore two independent instances of the Disk Driver, one for each disk. The Disk Driver is responsible for processing read and write requests submitted through the **`DiskRead`** and **`DiskWrite`** system calls.

Each request specifies the platter, the starting track, the starting sector, and the number of sectors to read or write. The Disk Driver uses this information to position the disk arm, read or write the requested sectors, and manage the process requesting the operation.

Requests are submitted to the Disk Driver through an internal queue. When a request arrives, it is added to the queue, and the driver determines which request to service next. The order in which requests are selected depends on the disk arm scheduling algorithm implemented by the driver. The four algorithms are:

- *First Come First Serve (FCFS)* services requests in the order they arrive.
- *Shortest Seek Time First (SSTF)* selects the request requiring the least arm movement from the current position.
- *Elevator* (or SCAN) moves the arm in one direction, servicing all requests in that direction, before reversing.
- *One Direction* (or C-SCAN) moves the arm in a single direction toward higher track numbers, returning to track 0 after reaching the highest track.

You ***must*** implement FCFS and SSTF to receive full credit for the project. Implementing a third algorithm (Elevator or One Direction) will earn bonus credit.

Each algorithm is selected at compile time by setting the DISK_ARM_ALG definition at the top of Devices.c to one of the constants defined in Devices.h:

```
/* In Devices.c -- change this value to select the active algorithm */
#define DISK_ARM_ALG   DISK_ARM_ALG_FCFS
```

The disk algorithm variable constants are defined in Devices.h:

```
/*
 * Disk arm scheduling algorithms.
 * Set DISK_ARM_ALG at the top of Devices.c to select the active algorithm.
 * Students must implement DISK_ARM_ALG_FCFS and at least one additional algorithm.
 */
#define DISK_ARM_ALG_FCFS           0   /* First Come First Serve              */
#define DISK_ARM_ALG_SSTF           1   /* Shortest Seek Time First            */
#define DISK_ARM_ALG_ELEVATOR       2   /* Elevator / SCAN (bidirectional)     */
#define DISK_ARM_ALG_ONE_DIRECTION  3   /* C-SCAN (one direction, then wrap)   */
```

Validation of your implementation requires that the algorithm choice it specified at compile time. Each submission will be assessed with the value set to `DISK_ARM_ALG_FCFS` and to `DISK_ARM_ALG_SSTF.` Be sure to specify in your submission any additional algorithms that you have implemented.

Once a request has been selected, the Disk Driver positions the arm if necessary. This is accomplished by calling **`device_control`** to move the arm to the desired track. Once the arm is positioned correctly, the driver issues individual sector read or write commands for the request. Each sector transfer is performed by calling **`device_control`** and then invoking **`waitdevice`** to block until the device signals completion through an interrupt.

Requests that span multiple tracks must be handled carefully. The driver is responsible for managing the transfer of sectors across track boundaries, including repositioning the arm when necessary. If other requests are pending for sectors on the destination track, the driver must decide whether to service those requests first or continue with the current multi-track operation. This decision may depend on the selected scheduling algorithm and the desired behavior of the driver.

When a request completes, the Disk Driver unblocks the requesting process, allowing it to resume execution. The driver then returns to the queue to select the next request and repeat the process.

All disk I/O in this project is asynchronous. User processes invoking **`DiskRead`** or **`DiskWrite`** are suspended until their request has been completed by the Disk Driver.

5. System Call Mapping

The sections below define the interface used for each of the system calls implemented in this project. For each of these functions, a return value of 0 indicates the operation was successful. If an error occurs during the call, a -1 value must be returned.

All system calls in this project must validate their input parameters before performing any operation. If any parameter is invalid, the call must return -1 immediately without queuing a request or blocking the calling process. The specific error conditions for each call are listed in the tables below.

SleepSeconds

```
int  SleepSeconds(int seconds);
```

CALL_ID == SYS_SLEEP

Input Argument	*arguments* Position
seconds	arguments[0]

Return Argument	*arguments* Position
Return value	arguments[3]

Error Conditions	Value
Invalid Parameter	-1

DiskRead

```
int  DiskRead(char* deviceName, void* dataBuffer, int platter, int track, int
firstSector, int sectors, int* status);
```

CALL_ID == SYS_DISKREAD

Input Argument	*arguments* Position
deviceName	arguments[0]
dataBuffer	arguments[1]
platter	arguments[2]
track	arguments[3]
firstSector	arguments[4]
sectors	arguments[5]

Return Argument	*arguments* Position
status	arguments[0]
Return value	arguments[3]

Error Conditions	Value

Invalid Parameter	-1

DiskWrite

```
int  DiskWrite(char* deviceName, void* dataBuffer, int platter, int track, int
firstSector, int sectors, int* status);
```

CALL_ID == SYS_DISKWRITE

Input Argument	*arguments* Position
deviceName	arguments[0]
dataBuffer	arguments[1]
platter	arguments[2]
track	arguments[3]
firstSector	arguments[4]
sectors	arguments[5]

Return Argument	*arguments* Position
status	arguments[0]
Return value	arguments[3]

Error Conditions	Value
Invalid Parameter	-1

DiskInfo

```
extern int  DiskInfo(char* deviceName, int* sectorSize, int* sectorCount, int*
trackCount, int* platterCount);
```

CALL_ID == SYS_DISKINFO

Input Argument	*arguments* Position
deviceName	arguments[0]

Return Argument	*arguments* Position
sectorSize	arguments[0]
sectorCount	arguments[1]
trackCount	arguments[2]
platterCount	arguments[4]
Return value	arguments[3]